CO1-AT1

NAME:M.POLI REDDY
REGNO:192325056

DSA0311-NATURAL LANGUAGE PROCESSING

Section-1:

Python-Based Resume Information Extraction System Using Regular Expressions

1. Aim

To design and implement a Python program using Regular Expressions (`re` module) to automatically extract candidate information from resumes in text format, generate structured candidate profiles, and shortlist candidates who meet the minimum eligibility criteria:

- Minimum 2 years of experience
- Must possess the Python skill

2. Algorithm

1. Read the resume text.
2. Extract the candidate's name using a regular expression.
3. Identify the email address.
4. Extract the mobile number.
5. Detect technical skills such as Python, Java, SQL, Machine Learning, and NLP.
6. Extract years of experience.
7. Generate a structured summary of each candidate.
8. Shortlist candidates who have at least 2 years of experience and Python as a skill.
9. Display the extracted profiles and eligible candidates.

3. Python Program

```
    import re


# Technical skills to search for
TECHNICAL_SKILLS = [
    "Python",
    "Java",
    "SQL",
    "Machine Learning",
    "NLP"
]


def extract_resume_information(resume):
```

```
    # Extract candidate name (assumes the first line contains the name)
    name_match = re.search(
        r"(?im)^\s*Name\s*:\s*(.+?)\s*$", resume
    )
    name = name_match.group(1).strip() if name_match else "Not Found"

    # Extract email address
    email_match = re.search(
        r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b",
        resume
    )
    email = email_match.group(0) if email_match else "Not Found"

    # Extract mobile number (10 digits, optional country code)
    mobile_match = re.search(
        r"(?<!\d)(?:\+91[-\s]?)?[6-9]\d{9}(?!\d)", resume
    )
    mobile = mobile_match.group(0) if mobile_match else "Not Found"

    # Detect technical skills (case-insensitive)
    skills = []
    for skill in TECHNICAL_SKILLS:
        pattern = r"(?<!\w)" + re.escape(skill) + r"(?!\w)"
        if re.search(pattern, resume, re.IGNORECASE):
            skills.append(skill)

    # Extract years of experience
    experience_match = re.search(
        r"(\d+(?:\.\d+)?)\s*\+?\s*years?\s+"
        r"(?:of\s+)?experience\b",
        resume,
        re.IGNORECASE
    )

    experience = (
        float(experience_match.group(1))
        if experience_match else 0.0
    )

    # Eligibility check
    eligible = experience >= 2 and "Python" in skills

    # Structured candidate profile
    profile = {
        "Name": name,
        "Email": email,
        "Mobile": mobile,
        "Technical Skills": skills,
        "Years of Experience": experience,
        "Eligible": "Yes" if eligible else "No"
    }

    return profile
```

```python
# Sample resumes
resumes = [
    """
    Name: M.POLIREDDY
    Email: poli.reddy@example.com
    Mobile: 9876543210
    Experience: 3 years of experience
    Skills: Python, SQL, Machine Learning
    """,

    """
    Name: Rahul Kumar
    Email: rahul.kumar@example.com
    Mobile: 9123456780
    Experience: 1 year of experience
    Skills: Python, Java
    """,

    """
    Name: saikumar
    Email: sai.reddy@example.com
    Mobile: 9988776655
    Experience: 5 years of experience
    Skills: Java, SQL, NLP
    """
]

# Process resumes
candidates = []

for resume in resumes:
    profile = extract_resume_information(resume)
    candidates.append(profile)

# Display all candidate profiles
print("===== CANDIDATE PROFILES =====")

for candidate in candidates:
    print("\n------------------------------")
    for key, value in candidate.items():
        print(f"{key}: {value}")

# Display eligible candidates
print("\n===== SHORTLISTED CANDIDATES =====")

shortlisted = [
    candidate for candidate in candidates
    if candidate["Eligible"] == "Yes"
]

if shortlisted:
    for candidate in shortlisted:
        print(
            f"Name: {candidate['Name']}, "
```

```
            f"Experience: {candidate['Years of Experience']} years, "
            f"Skills: {', '.join(candidate['Technical Skills'])}"
        )
else:
    print("No eligible candidates found.")
```

4. Sample Output

```
print("No eligible candidates found.")

===== CANDIDATE PROFILES =====

------------------------------
Name: M.POLIREDDY
Email: poli.reddy@example.com
Mobile: 9876543210
Technical Skills: ['Python', 'SQL', 'Machine Learning']
Years of Experience: 3.0
Eligible: Yes

------------------------------
Name: Rahul Kumar
Email: rahul.kumar@example.com
Mobile: 9123456780
Technical Skills: ['Python', 'Java']
Years of Experience: 1.0
Eligible: No

------------------------------
Name: saikumar
Email: sai.reddy@example.com
Mobile: 9988776655
Technical Skills: ['Java', 'SQL', 'NLP']
Years of Experience: 5.0
Eligible: No

===== SHORTLISTED CANDIDATES =====
Name: M.POLIREDDY, Experience: 3.0 years, Skills: Python, SQL, Machine Learning
```

5. Explanation of Regular Expressions

Regular expression	Purpose
`(?im)^\s*Name\s*:\s*(.+?)\s*$`	Extracts a name from a line beginning with `Name:`
`\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b`	Identifies email addresses
`(?<!\d)(?:\+91[-\s]?)?[6-9]\d{9}(?!\d)`	Matches common Indian 10-digit mobile numbers, optionally prefixed by `+91`
`(\d+(?:\.\d+)?)\s*\+?\s*years?\s+(?:of\s+)?experience\b`	Extracts numeric experience values such as `3 years of experience`

`(?<!\w)Python(?!\w)`	Detects Python as a separate word rather than part of another word

6. Eligibility Criteria

A candidate is shortlisted only when both conditions are true:

python
```
experience >= 2 and "Python" in skills
```

Candidate	Experience	Python skill	Result
polireddy	3 years	Yes	Eligible
Rahul Kumar	1 year	Yes	Not eligible
saikumar	5 years	yes	eligible

7. Result

The Python-based resume information extraction system successfully demonstrates how Regular Expressions can be used to extract candidate names, email addresses, mobile numbers, technical skills, and years of experience. It generates structured candidate profiles and automatically shortlists candidates who have at least 2 years of experience and the Python skill.

Note: This implementation assumes resumes contain a labeled `Name:` field and experience in a recognizable format. For real-world recruitment, resume formats vary, so the extraction patterns should be expanded and validated before using the system for actual hiring decisions.

Section-2:

Python-Based Product Search System Using Regular Expressions

1. Aim

To design and implement a Python-based product search system using the `re` module that supports flexible keyword searches and generates a report showing the matching products and the total number of matches for each search.

2. Algorithm

1. Create a list of product names.
2. Accept a search keyword and search type.
3. Use Regular Expressions to perform the selected search:
    - Exact keyword search
    - Prefix-based search
    - Suffix-based search
    - Partial keyword search
    - Case-insensitive search
4. Display all matching product names.
5. Count the matching products.
6. Generate a search report containing the search type, keyword, matching products, and total count.

3. Python Program

```
import re

# Sample product database
products = [
    "Apple iPhone 15",
    "Samsung Galaxy S24",
    "Apple MacBook Air",
    "HP Laptop",
    "Dell Laptop",
    "Wireless Mouse",
    "Bluetooth Headphones",
    "Apple Watch",
    "Samsung Television",
    "Logitech Keyboard"
]


def search_products(keyword, search_type):
    matches = []

    # Escape special regex characters in the keyword
    keyword = re.escape(keyword)

    for product in products:

        if search_type == "exact":
            # Match the complete product name
            pattern = r"^" + keyword + r"$"

        elif search_type == "prefix":
            # Match products starting with the keyword
            pattern = r"^" + keyword

        elif search_type == "suffix":
            # Match products ending with the keyword
            pattern = keyword + r"$"
```

```
        elif search_type == "partial":
            # Match the keyword anywhere in the product name
            pattern = keyword

        elif search_type == "case-insensitive":
            # Match the keyword regardless of letter case
            pattern = keyword

        else:
            print("Invalid search type!")
            return []

        # Apply case-insensitive matching for all searches
        if re.search(pattern, product, re.IGNORECASE):
            matches.append(product)

    return matches


def generate_report(keyword, search_type):
    matches = search_products(keyword, search_type)

    print("\n========== SEARCH REPORT ==========")
    print("Search Type:", search_type)
    print("Keyword:", keyword)

    if matches:
        print("Matching Products:")
        for product in matches:
            print("-", product)
    else:
        print("No matching products found.")

    print("Total Matching Products:", len(matches))
    print("===================================")


# Demonstrate all five search types
generate_report("Apple iPhone 15", "exact")
generate_report("Apple", "prefix")
generate_report("Laptop", "suffix")
generate_report("phone", "partial")
generate_report("sAmSuNg", "case-insensitive")
```

4. Explanation of Regular Expressions

Search type	Regular expression	Purpose
Exact keyword	`^keyword$`	Matches the entire product name
Prefix search	`^keyword`	Matches names beginning with the keyword
Suffix search	`keyword$`	Matches names ending with the keyword

Partial search	`keyword`	Matches the keyword anywhere in the name
Case-insensitive	`keyword` with `re.IGNORECASE`	Matches regardless of uppercase or lowercase letters

Special symbols:

- `^` — beginning of a string.
- `$` — end of a string.
- `re.search()` — searches for a matching pattern.
- `re.IGNORECASE` — ignores differences in letter case.
- `re.escape()` — treats special characters in user input as literal characters rather than regex operators.

5. Sample Output

```
...
    ========== SEARCH REPORT ==========
    Search Type: exact
    Keyword: Apple iPhone 15
    Matching Products:
    - Apple iPhone 15
    Total Matching Products: 1
    ==================================

    ========== SEARCH REPORT ==========
    Search Type: prefix
    Keyword: Apple
    Matching Products:
    - Apple iPhone 15
    - Apple MacBook Air
    - Apple Watch
    Total Matching Products: 3
    ==================================

    ========== SEARCH REPORT ==========
    Search Type: suffix
    Keyword: Laptop
    Matching Products:
    - HP Laptop
    - Dell Laptop
    Total Matching Products: 2
    ==================================

    ========== SEARCH REPORT ==========
    Search Type: partial
    Keyword: phone
    Matching Products:
    - Apple iPhone 15
    - Bluetooth Headphones
    Total Matching Products: 2
    ==================================

    ========== SEARCH REPORT ==========
    Search Type: case-insensitive
    Keyword: sAmSuNg
    Matching Products:
    - Samsung Galaxy S24
    - Samsung Television
    Total Matching Products: 2
    ==================================
```

6. Expected Results Summary

Search operation	Keyword	Number of matches
Exact	`Apple iPhone 15`	1
Prefix	`Apple`	3
Suffix	`Laptop`	2
Partial	`phone`	2
Case-insensitive	`sAmSuNg`	2

7. Result

The Python product search system successfully performs exact, prefix, suffix, partial, and case-insensitive searches using Regular Expressions. It displays all matching product names and generates a report showing the total number of matching products for each search.

Note: In this implementation, all searches are case-insensitive for convenience. Exact matching means the entire product name must match the keyword, while prefix, suffix, and partial matching search the corresponding portion of the product name.

Section:3

Python-Based Student Registration Validation System Using Regular Expressions

1. Aim

To design and implement a Python program using the `re` module to validate student registration details before course enrollment and generate a final registration status report.

The system validates:

- Student register number
- Institutional email address
- Course code
- Semester information
- Student mobile number

2. Algorithm

1. Start the program.

2. Accept student registration details.
3. Validate the register number using a regular expression.
4. Validate the institutional email address.
5. Validate the course code.
6. Validate the semester information.
7. Validate the mobile number.
8. Display a success or failure message for each field.
9. Generate the final registration status report.
10. Declare registration successful only if all fields are valid.

3. Python Program

```
import re


def validate_register_number(register_number):
    pattern = r"^\d{2}[A-Z]{2}\d{3}$"
    return bool(re.fullmatch(pattern, register_number))


def validate_email(email):
    pattern = r"^[A-Za-z0-9._%+-]+@university\.edu$"
    return bool(re.fullmatch(pattern, email, re.IGNORECASE))


def validate_course_code(course_code):
    pattern = r"^[A-Z]{2,4}\d{3}$"
    return bool(re.fullmatch(pattern, course_code))


def validate_semester(semester):
    pattern = r"^[1-8]$"
    return bool(re.fullmatch(pattern, semester))


def validate_mobile(mobile):
    pattern = r"^[6-9]\d{9}$"
    return bool(re.fullmatch(pattern, mobile))


# Get student details
register_number = input("Enter Register Number: ").strip()
email = input("Enter Institutional Email: ").strip()
course_code = input("Enter Course Code: ").strip().upper()
semester = input("Enter Semester (1-8): ").strip()
mobile = input("Enter Mobile Number: ").strip()


# Validate each field
results = {
    "Register Number": validate_register_number(register_number),
```

```
    "Institutional Email": validate_email(email),
    "Course Code": validate_course_code(course_code),
    "Semester": validate_semester(semester),
    "Mobile Number": validate_mobile(mobile)
}


# Display validation messages
print("\n===== VALIDATION REPORT =====")

for field, is_valid in results.items():
    if is_valid:
        print(f"{field}: Valid")
    else:
        print(f"{field}: Invalid")

# Generate final registration status
print("\n===== REGISTRATION STATUS =====")

if all(results.values()):
    print("Registration Successful!")
    print("All student details are valid.")
else:
    print("Registration Failed!")
    print("Please correct the invalid fields and try again.")
```

4. Regular Expressions Used

Field	Regular expression	Validation rule
Register number	`^\d{2}[A-Z]{2}\d{3}$`	Two digits, two uppercase letters, three digits
Institutional email	`^[A-Za-z0-9._%+-]+@university\.edu$`	Must end with `@university.edu`
Course code	`^[A-Z]{2,4}\d{3}$`	Two to four letters followed by three digits
Semester	`^[1-8]$`	A single digit from 1 to 8
Mobile number	`^[6-9]\d{9}$`	A 10-digit Indian mobile number beginning with 6–9

Meaning of important regex symbols

- `^` — beginning of the string.
- `$` — end of the string.
- `\d` — any digit from 0 to 9.
- `[A-Z]` — any uppercase English letter.

- `{2,4}` — between two and four repetitions.
- `{3}` — exactly three repetitions.
- `+` — one or more repetitions.
- `re.fullmatch()` — ensures the entire input follows the specified pattern.

5. Sample Input



8. Final Registration Status Logic

The program uses:

python

```
if all(results.values()):
    print("Registration Successful!")
else:
    print("Registration Failed!")
```

The `all()` function returns `True` only when every field passes validation.

9. Result

The Python-based student registration validation system successfully validates the register number, institutional email, course code, semester, and mobile number using Regular Expressions. It displays appropriate validation messages and generates a final registration report. Registration is successful only when all five fields are valid.

Note: These patterns are illustrative. A real university should adjust them to its official register number, email domain, course code, semester, and phone-number policies. Regex checks formatting only; it does not confirm that a student is enrolled, that a course exists, or that an email or phone number is genuine.